

PROGRAMMING · BEGINNER

C++ Essentials

Write clearer C++ with objects and ownership.



Free C++ foundations covering modern syntax, OOP, STL and practical problem solving.



6-8 weeks



4 modules



Free course

[Explore the learning experience →](#)

Practical learning. Clear progression. Work you can explain.

YOUR NEXT CAPABILITY

What will you be able to do?

Progress from C++ language fundamentals into classes, containers, algorithms, memory ownership and a portfolio-ready console application.



Write modern C++ programs



Apply object-oriented design



Use STL containers and algorithms



Understand memory and ownership basics



Who this is for

- C learners progressing to C++
- Engineering students
- Aspiring systems developers



Your starting point

- Basic programming familiarity is helpful

Your progression at a glance

FROM CONCEPT TO EVIDENCE



Learn it. Apply it. Explain it.

Move from guided concepts to practical exercises and a coherent piece of work. Use the course resources to revisit difficult ideas and make your reasoning visible.

THE CURRICULUM

A clear route through the subject.

Every module builds towards practical work. The course outline below is aligned with the academy's current program catalog.

1 Modern C++ Foundations

Types, functions, references and clean program structure.

- Toolchain and first program
- Types and control flow
- Functions and references
- Namespaces and headers

2 Object-Oriented C++

Classes, encapsulation and polymorphism.

- Classes and objects
- Constructors
- Inheritance
- Interfaces and polymorphism

3 STL and Problem Solving

Containers, iterators and algorithms.

- Vectors and maps
- Iterators
- Algorithms
- Exceptions

4 Ownership and Capstone

Memory, RAII and a complete application.

- Stack and heap
- Smart pointers
- RAII
- Console application capstone

FROM CONCEPT TO EVIDENCE



{ } Practice with purpose

Model a practical problem with classes, use standard containers and algorithms, and explain resource ownership in your solution.

BUILD SOMETHING THAT DEMONSTRATES YOUR LEARNING

A modern C++ console application

Model a practical problem with classes, use standard containers and algorithms, and explain resource ownership in your solution.

**Class and ownership design****Container and algorithm examples****Tested console project**

Tools and working methods

C++20 compiler

VS Code

Git

Tools are part of the learning workflow, not partner endorsements. Examples may evolve while preserving the learning outcomes.



Delivery

Self-paced online · 6–8 weeks

Confirm the current schedule, delivery arrangements and any prerequisites before enrolling.



Completion

A completion certificate is available after the required course work and a passing assessment.

Course completion is not a job, placement or funding guarantee.

Your next steps

- 1 Review the curriculum and your starting point.
- 2 Create your learner account and enroll for free.
- 3 Start learning, keep your evidence and track your progress.

[Open the course experience →](#)

Customer-preview edition. Interactive links open the local academy prototype. Confirm course availability, schedule and commercial terms with the academy.

INSIDE THE LEARNING EXPERIENCE

Represent ownership with values and containers

A study-planner application needs a list of tasks. Each task has a title and a completion flag. Users can add tasks while the program is running.

Understand the concept

`std::vector` owns its elements and manages its allocated storage. Prefer this value-based model to manually allocating each task. Iterate with const references when you only need to inspect tasks. Growing a vector may reallocate its storage, so do not keep references to elements across an insertion that might invalidate them.

Worked example

```
struct Task { std::string title; bool done; };
std::vector<Task> tasks{{"Review vectors", false}};
tasks.push_back({"Write tests", true});
for (const auto& task : tasks) {
    std::cout << task.title << ": " << task.done << "\n";
}
```

Interpret the result

The vector contains two independently owned task values. The range loop reads them without copying each task. The container releases its storage when its lifetime ends.

Knowledge check

Why might a saved reference become invalid after `push_back`?

Explanation: If the vector reallocates, its elements move to new storage and references into the old storage are invalidated.

YOUR PRACTICAL ASSIGNMENT

Build evidence, not just familiarity.

Represent ownership with values and containers



Your task

- Add a function that counts completed tasks.
- Implement lookup by a stable task ID instead of a saved element reference.
- Save and load task records with error handling for malformed input.



Submission review criteria

- Clear ownership without raw owning pointers
- Correct empty-list and missing-ID behavior
- Tests showing state survives save and reload

Submit

A working artifact or analysis, a short explanation of your decisions, and evidence that addresses each review criterion.

Reflect

Describe one failed approach, how you diagnosed it, and what you would test next. Identify limitations instead of hiding them.

Your project connection

Model a practical problem with classes, use standard containers and algorithms, and explain resource ownership in your solution.

Use the sample assignment as a stepping stone toward the course project, not as a substitute for the full curriculum.

[Explore the worked lesson](#) →

Proposed teaching sample for curriculum and UX review. Full lesson production, instructor review and delivery scheduling remain separate work.